Trabajo Práctico N.º 7 · Migración a MongoDB

Proyecto Food Store · Ciclo Lectivo 2026 · Comisión 2PRO3

Integrantes: Bruno Fiouchetta, Elías Tello, Belén Calvo, Hernán González

Repositorio: github.com/belcalvo93/BaseDatos2-MongoDB

Consigna 1: Modelo Relacional de Origen y Diseño Documental (Bruno Fiouchetta)

seleccion_pedidos.sql · modelo_documental.md

1. Tablas y restricciones intervinientes en PostgreSQL (bd2_tp3)

La reconstrucción integral del pedido en el esquema normalizado (3FN) involucra cuatro tablas vinculadas por claves foráneas:

Tabla	Clave Primaria (PK)	Claves Foráneas (FK)	Restricciones y Reglas de Negocio Clave	Columnas Migradas
pedido	`id_pedido` (BIGINT)	`id_cliente` → `cliente(id_cliente)`	`forma_pago` tipo enum (`'EFECTIVO'`, `'TARJETA'`, `'TRANSFERENCIA'`). No posee columna `estado`.	`id_pedido, fecha, forma_pago, id_cliente`
detalle_pedido	`id_detalle` (BIGINT)	`id_pedido` → `pedido` (CASCADE) `id_producto` → `producto` (RESTRICT)	**UNIQUE (id_pedido, id_producto)** (impide duplicar productos en un mismo pedido). `CHECK (cantidad > 0)`, `CHECK (precio_unitario >= 0)`. Subtotal no persistido.	`id_pedido, id_producto, cantidad, precio_unitario`
cliente	`id_cliente` (BIGINT)	—	`UNIQUE (email)` (clave candidata).	`id_cliente, nombre, apellido, email`
producto	`id_producto` (BIGINT)	`id_categoria` → `categoria`	`CHECK (precio >= 0)`. Catálogo con precio actual dinámico y baja lógica (`activo`).	`id_producto, nombre`

2. Esquema BSON de la colección pedidos en MongoDB

Se consolidó la estructura relacional dispersa en una única colección autocontenida llamada **`foodstore.pedidos`**, con cliente y detalle embebidos:

```
{
  "_id": 1,                                    // = id_pedido
  "fecha": ISODate("2026-08-01T13:15:00.000Z"),  // UTC
  "forma_pago": "EFECTIVO",                    // "estado"
  "total": NumberDecimal("9987.91"),           // auditado
  "cliente": {                                 // embebido 1:1
    "id_cliente": 1,
    "nombre": "Ana Gutiérrez",
    "email": "ana.gutierrez@example.com"
  },
  "lineas": [                                  // arreglo 1:N
    {
      "id_producto": 1,
      "nombre": "Pan de campo 1kg",
      "cantidad": 2,
      "precio_unitario": NumberDecimal("2200.00")  // R4
    }
  ]
}
```

Decisiones fundamentales de diseño:

- **Conservación de Identificadores:** Se conservó `_id = id_pedido`, `id_cliente` e `id_producto` para auditoría directa contra PostgreSQL.
- **Embebido de cliente:** Relación 1:1 donde viajan los datos esenciales de facturación sin necesidad de joins.
- **Embebido de líneas:** Cardinalidad baja (1 a 6 renglones). La existencia de las líneas depende enteramente del pedido.
- **Total persistido:** Siguiendo las directivas de clase, se guarda `total` en `Decimal128` y se audita con agregaciones frente a la suma de líneas.
- **Subtotal calculado:** Cada línea omite el subtotal para evitar dependencias transitivas; se evalúa al vuelo con `$multiply`.

3. Selección representativa de 8 pedidos desde bd2_tp3 (seleccion_pedidos.sql)

Se seleccionaron **8 pedidos reales** (30 líneas de detalle, suma acumulada de **$84.403,74** y promedio de **$10.550,47**), cubriendo todos los requisitos:

id_pedido (_id)	Fecha (UTC)	Forma de Pago	Cliente	Líneas	Total Acumulado	IDs Productos en lineas[] (Criterios evaluados)
1	2026-08-01 13:15:00	EFECTIVO	Ana Gutiérrez	5	$9.987,91	**1** (precio $2200), 5, 66076, 72206, 83667
2	2026-08-03 21:40:00	TARJETA	Bruno Salinas	6	$19.036,35	3, 7, 38558, 51872, 66351, 69939 (Supera umbral d)

id_pedido (_id)	Fecha (UTC)	Forma de Pago	Cliente	Líneas	Total Acumulado	IDs Productos en lineas[] (Criterios evaluados)
3	2026-08-07 12:05:00	TRANSFERENCIA	Ana Gutiérrez	6	$15.453,62	2, 6, 67973, 71591, 71682, 72234 (Supera umbral d)
4	2026-08-12 23:30:00	TARJETA	Carla Medina	5	$16.658,04	8, 9, 66290, 68909, 81323 (Supera umbral d)
5	2026-08-18 15:00:00	EFECTIVO	Diego Ferrari	3	$4.293,65	4, 75467, 80098
45018	2025-05-23 06:26:12	EFECTIVO	Nombre4545 Apellido4545	1	$1.931,96	35767 (Caso sugerido en clase: pedido de 1 línea)
59428	2025-05-08 10:01:17	TRANSFERENCIA	Nombre4898 Apellido4898	2	$7.552,58	**1** (precio $2500), 64780
74364	2026-02-27 12:13:59	TARJETA	Nombre6217 Apellido6217	2	$9.489,63	**1** (precio $2500), 35739

Cumplimiento de consigna: Presenta las 3 formas de pago (EFECTIVO: 3, TARJETA: 3, TRANSFERENCIA: 2); producto testigo repetido (`id_producto = 1`) en 3 pedidos con precio histórico congelado R4 ($2200 en pedido 1 vs $2500 en 59428 y 74364); y variedad de líneas (1 a 6) y totales ($1931 a $19036).

Consignas 2 y 3: Migración, Validación e Integridad (Elías Tello)

consultas_mongodb.js

1. Creación de la Colección y Validador Estricto de Esquema

Para asegurar integridad a nivel de base de datos sin depender exclusivamente del cliente, la colección `foodstore.pedidos` se creó con un validador compuesto que combina `$jsonSchema` y expresiones lógicas `$expr` (validationLevel: 'strict', validationAction: 'error'):

```
db = db.getSiblingDB("foodstore");
db.createCollection("pedidos", { validationLevel: "strict", validationAction: "error", validator: { $and: [
  { $jsonSchema: { bsonType: "object", required: ["_id", "fecha", "forma_pago", "total", "cliente", "lineas"],
      properties: {
        _id: { bsonType: ["int", "long"] },  fecha: { bsonType: "date" },  total: { bsonType: "decimal", minimum: 0
},
        forma_pago: { enum: ["EFECTIVO", "TARJETA", "TRANSFERENCIA"] },
        cliente: { bsonType: "object", required: ["id_cliente", "nombre"], properties: {
          id_cliente: { bsonType: ["int", "long"] }, nombre: { bsonType: "string" }, email: { bsonType: "string" } }
},
        lineas: { bsonType: "array", minItems: 1, items: { bsonType: "object",
          required: ["id_producto", "nombre", "cantidad", "precio_unitario"], properties: {
            id_producto: { bsonType: ["int", "long"] },  nombre: { bsonType: "string" },
            cantidad: { bsonType: "int", minimum: 1 },  precio_unitario: { bsonType: "decimal", minimum: 0 } } } } } }
} },
  // R2: ningún id_producto repetido dentro del mismo pedido
  { $expr: { $eq: [ { $size: "$lineas.id_producto" }, { $size: { $setUnion: ["$lineas.id_producto", []] } } ] } }
] } });
```

2. Reglas de Integridad Implementadas

R1. Precio histórico congelado (Regla R4 de Food Store):
En PostgreSQL, `detalle_pedido.precio_unitario` almacena el precio al que se vendió el ítem. En MongoDB, cada subdocumento de `lineas` guarda ese importe exacto con tipado `Decimal128` (`NumberDecimal`), evitando errores de redondeo binario. El valor queda totalmente desvinculado del catálogo relacional (`producto.precio`).
Prueba fehaciente: El `id_producto = 1` se cargó a **$2.200,00** en el pedido 1 y a **$2.500,00** en los pedidos 59428 y 74364.

R2. Unicidad de producto dentro del pedido:
En PostgreSQL lo garantiza la restricción compuesta `UNIQUE (id_pedido, id_producto)`.
En MongoDB, como los índices únicos multikey no aplican a nivel intra-documento (sino entre documentos), se utilizó la regla de validación de conjuntos `$setUnion`.
Mecanismo: Si la cantidad de elementos en `lineas.id_producto` difiere del tamaño de su unión de conjuntos únicos, el motor aborta la operación con código de error 121 (Document failed validation).

3. Justificación de Diseño: Embeber vs. Colecciones Separadas

- **Cliente (objeto embebido):** Relación 1:1. El pedido requiere disponer inmediatamente del nombre y correo del comprador para visualización y emisión de comprobantes. Se embeben solo los atributos estables; el maestro de clientes continúa residiendo en su tabla relacional.
- **Líneas (arreglo de subdocumentos):** Las líneas poseen una relación de contención existencial con el pedido (en PostgreSQL tienen `ON DELETE CASCADE`) y carecen de sentido de forma autónoma. Al ser un número reducido (1 a 6

renglones), no existe riesgo de aproximarse al límite de 16 MB por documento BSON.
- **Rendimiento y Atomicidad:** Al embeber ambos elementos, un pedido completo se lee y se escribe en **una única operación de E/S atómica**, sin necesidad de `$lookup`.

4. Ejecución de Migración y Verificación de Integridad

Se ejecutó `insertMany()` migrando los 8 pedidos. La auditoría cruzada contra PostgreSQL arrojó coincidencia exacta en los 8 pedidos:

```
// Salida de ejecución en mongosh (evidencias/salida_migracion_y_verificacion.txt):
Documentos en foodstore.pedidos: 8
1     | EFECTIVO      | lineas=5 | total=9987.91  | suma_lineas=9987.91  | coincide=true
2     | TARJETA       | lineas=6 | total=19036.35 | suma_lineas=19036.35 | coincide=true
3     | TRANSFERENCIA | lineas=6 | total=15453.62 | suma_lineas=15453.62 | coincide=true
4     | TARJETA       | lineas=5 | total=16658.04 | suma_lineas=16658.04 | coincide=true
5     | EFECTIVO      | lineas=3 | total=4293.65  | suma_lineas=4293.65  | coincide=true
45018 | EFECTIVO      | lineas=1 | total=1931.96  | suma_lineas=1931.96  | coincide=true
59428 | TRANSFERENCIA | lineas=2 | total=7552.58  | suma_lineas=7552.58  | coincide=true
74364 | TARJETA       | lineas=2 | total=9489.63  | suma_lineas=9489.63  | coincide=true
Pedidos con total distinto de la suma de líneas: 0 (esperado: 0)
OK R2: rechazado el pedido con producto repetido (código 121: Document failed validation)
```

Nota técnica de encoding: En `bd2_tp3` el apellido de Ana Gutiérrez figuraba corrupto como 'GutiÃ©rrez' (falla histórica UTF-8 importada como Latin-1). Se normalizó a 'Gutiérrez' mediante `convert_from(convert_to(apellido, 'LATIN1'), 'UTF8')` sin alterar IDs ni montos.

Consigna 4: Consultas de Negocio en MongoDB (Belén Calvo)

consultas_mongodb.js

Criterios Generales de Consulta y Aclaraciones Previas

1. Criterio sobre 'eliminado = false': Ninguna de las tablas de origen (`pedido`, `detalle_pedido`, `cliente`, `producto`) posee columna de borrado lógico en `bd2_tp3`. La columna `producto.activo` corresponde al estado actual del catálogo, no de la venta histórica, por lo que no se aplica ningún filtro de eliminación para preservar fielmente los datos.
2. Equivalencia de Estado: La tabla `pedido` no tiene columna `estado`; se emplea `forma_pago` (EFECTIVO, TARJETA, TRANSFERENCIA) como atributo de clasificación según se acordó en el diseño.

Consultas Implementadas y Resultados Obtenidos

a) Listado completo de pedidos (Identificador y Estado)

```
db.pedidos.find({}, { _id: 1, forma_pago: 1 }).sort({ _id: 1 });
// Resultado: 8 documentos retornados [1: EFECTIVO, 2: TARJETA, 3: TRANSFERENCIA, 4: TARJETA, 5: EFECTIVO, 45018:
EFECTIVO, 59428: TRANSFERENCIA, 74364: TARJETA]. Cantidad: 8
```

b) Pedidos filtrados por forma_pago = 'EFECTIVO'

Se seleccionó 'EFECTIVO' dado que recupera un subconjunto no trivial de 3 pedidos sobre los 8 totales, demostrando la selectividad del filtro:

```
db.pedidos.find(
  { forma_pago: "EFECTIVO" },
  { _id: 1, fecha: 1, forma_pago: 1, total: 1 }
).sort({ _id: 1 });
```

```
// Salida de ejecución:
1     | 2026-08-01T13:15:00.000Z | EFECTIVO | 9987.91
5     | 2026-08-18T15:00:00.000Z | EFECTIVO | 4293.65
45018 | 2025-05-23T06:26:12.027Z | EFECTIVO | 1931.96
Cantidad de documentos: 3
```

c) Pedidos que contienen id_producto = 1 (Pan de campo 1kg)

Se consulta por el identificador único numérico del producto en el arreglo embebido. El operador de proyección posicional `"lineas.$"` permite aislar la línea coincidente para inspeccionar el precio congelado (regla R4):

```
db.pedidos.find(
  { "lineas.id_producto": 1 },
  { _id: 1, forma_pago: 1, "lineas.$": 1 }
).sort({ _id: 1 });
```

```
// Salida de ejecución:
1     | EFECTIVO      | Pan de campo 1kg | cant=2 | precio_unit=2200.00
59428 | TRANSFERENCIA | Pan de campo 1kg | cant=3 | precio_unit=2500.00
74364 | TARJETA       | Pan de campo 1kg | cant=3 | precio_unit=2500.00
Cantidad de documentos: 3
```

d) Pedidos con total > $10.000 (orden descendente por total)

Justificación del umbral elegido ($10.000): El total promedio de los 8 pedidos migrados es de **$10.550,47**. El umbral de $10.000,00 segmenta limpiamente el lote: 3 pedidos se sitúan por encima (2, 4 y 3) y 5 por debajo. Destaca el pedido 1 ($9.987,91), que queda inmediatamente debajo del límite, demostrando que la comparación aritmética con `NumberDecimal` es exacta al centavo. Al estar persistido el total, la consulta se resuelve con un `find` directo sin agregación:

```
db.pedidos.find(
  { total: { $gt: NumberDecimal("10000") } },
  { _id: 1, forma_pago: 1, total: 1 }
).sort({ total: -1 });
```

```
// Salida de ejecución:
2 | TARJETA       | 19036.35
4 | TARJETA       | 16658.04
3 | TRANSFERENCIA | 15453.62
Cantidad de documentos: 3
```

Auditoría de Redundancia Controlada: Total Guardado vs. Suma de Líneas

Siguiendo las directivas pedagógicas de cátedra, se implementó una consulta de agregación para comprobar que el campo `total` persistido coincida estrictamente con la suma calculada en el arreglo `lineas`:

```
const descuadrados = db.pedidos.aggregate([
  {
    $project: {
      total: 1,
      suma_lineas: { $sum: { $map: { input: "$lineas", as: "l", in: { $multiply: ["$$l.cantidad",
"$$l.precio_unitario"] } } } }
    }
  },
  { $match: { $expr: { $ne: ["$total", "$suma_lineas"] } } }
]).toArray();
// Resultado: Pedidos con total distinto de la suma de líneas: 0 (esperado: 0)
```

Consigna 5: Verificación Cruzada PostgreSQL vs. MongoDB (Belén Calvo)

consultas_postgresql.sql · consigna5_tabla_comparativa.md

1. Consultas Equivalentes en PostgreSQL (bd2_tp3)

Para asegurar una comparación rigurosa, todas las consultas SQL se limitaron exactamente a los 8 pedidos migrados mediante la cláusula `WHERE p.id_pedido IN (1, 2, 3, 4, 5, 45018, 59428, 74364)`. Sin esta restricción, PostgreSQL operaría sobre la totalidad de la base masiva y los volúmenes no serían equiparables.

Consulta b) en PostgreSQL (forma_pago = 'EFECTIVO')

```
SELECT p.id_pedido, p.fecha, p.forma_pago,
       SUM(dp.cantidad * dp.precio_unitario) AS total
FROM pedido p
INNER JOIN detalle_pedido dp ON dp.id_pedido =
p.id_pedido
WHERE p.id_pedido IN (1, 2, 3, 4, 5, 45018, 59428,
74364)
  AND p.forma_pago = 'EFECTIVO'
GROUP BY p.id_pedido, p.fecha, p.forma_pago
ORDER BY p.id_pedido;
```

Consulta c) en PostgreSQL (id_producto = 1)

```
SELECT p.id_pedido, p.forma_pago, pr.nombre,
       dp.cantidad, dp.precio_unitario
FROM pedido p
INNER JOIN detalle_pedido dp ON dp.id_pedido =
p.id_pedido
INNER JOIN producto pr       ON pr.id_producto =
dp.id_producto
WHERE p.id_pedido IN (1, 2, 3, 4, 5, 45018, 59428,
74364)
  AND dp.id_producto = 1
ORDER BY p.id_pedido;
```

2. Tabla Comparativa Oficial de Verificación

Consulta Evaluada	Cantidad en PostgreSQL	Cantidad en MongoDB	IDs Obtenidos	Coinciden IDs	Coinciden Importes y Precios
b) Estado / Forma de pago: EFECTIVO	3	3	1, 5, 45018	SÍ	SÍ ($9987.91, $4293.65, $1931.96)
c) Producto: id_producto = 1 (Pan de campo)	3	3	1, 59428, 74364	SÍ	SÍ ($2200.00 en 1; $2500.00 en resto)
Control adicional: a) Todos los pedidos	8	8	1, 2, 3, 4, 5, 45018, 59428, 74364	SÍ	SÍ (Total acumulado: $84.403,74)
Control adicional: d) Total > $10.000 (descendente)	3	3	2, 4, 3 (mismo orden)	SÍ	SÍ ($19036.35, $16658.04, $15453.62)

3. Análisis Técnico de Diferencias Detectadas

1. Precisión de las Marcas Temporales (Timestamps):
En los pedidos procedentes de la carga masiva (45018, 59428, 74364), PostgreSQL almacena microsegundos en su tipo `TIMESTAMPTZ` (ej. `2025-05-23 06:26:12.027438+00`), mientras que el tipo `Date` de BSON en MongoDB trabaja internamente con precisión de milisegundos (ej. `2025-05-23T06:26:12.027Z`). En consecuencia, la migración trunca los últimos 3 dígitos de la fracción de segundo. No altera IDs, importes ni resultados de filtros temporales a nivel de segundo/minuto.

2. Eliminación de Joins y Complejidad de Consulta:
• *En la consulta b):* PostgreSQL debe computar `SUM(cantidad * precio_unitario)` y ejecutar `GROUP BY` requiriendo un `INNER JOIN` entre `pedido` y `detalle_pedido`. En MongoDB, al tener `total` persistido como redundancia auditada, se lee directamente del documento en un solo acceso.
• *En la consulta c):* PostgreSQL requiere obligatoriamente dos `INNER JOIN` uniendo 3 tablas (`pedido`, `detalle_pedido` y `producto`) para asociar el nombre y el ítem. En MongoDB, alcanza con un `find({"lineas.id_producto": 1})` directo, porque las líneas están embebidas en el pedido.

3. Ausencia de Borrado Lógico:
Se ratifica que ninguna tabla de pedidos o detalles en `bd2_tp3` cuenta con atributos de baja lógica; los 8 pedidos reflejan operaciones en firme.

Consignas 6 y 7: Inteligencia Artificial y Conclusión Técnica (Hernán González)

consigna6_ia_contraste.md · prueba_indice_unico_ia.js

Consigna 6: Consulta a la IA y Contraste Crítico

Prompt a la IA: Se formuló un prompt explicitando: 1) Modelo relacional (4 tablas normalizadas en 3FN con `UNIQUE(id_pedido, id_producto)`); 2) Patrón de acceso frecuente a pedidos completos; y 3) Necesidad de mantener coherentes precios y cantidades con precio congelado R4.

Respuesta resumida de la IA: Recomendó embeber cliente (parcial) y líneas (en arreglo); congelar `precio_unitario` como escalar; crear un índice único `{"lineas.id_producto": 1}` para evitar productos repetidos en el pedido; y guardar `subtotal` y `total` con punto flotante.

Contraste Crítico 1: Restricción de Unicidad de Productos dentro del Pedido

Afirmación IA: *'Crear un índice único sobre el arreglo embebido: db.pedidos.createIndex({"lineas.id_producto": 1}, {unique: true})'.*
Veredicto: FALSO / ALUCINACIÓN CRÍTICA.
Contraste oficial y evidencia de laboratorio (prueba_indice_unico_ia.js): La documentación oficial de MongoDB (*Unique Indexes - Unique Constraint Across Separate Documents*) aclara que un índice único impide valores idénticos **entre documentos distintos**, pero **no dentro de los elementos de un mismo arreglo**. Se ejecutó la prueba en `mongosh` confirmando ambas consecuencias catastróficas:
1. *Falso positivo de integridad:* Un pedido con dos líneas del producto 1 fue **ACEPTADO** (no impide el duplicado interno).
2. *Bloqueo del sistema:* Un segundo pedido legítimo con el producto 1 fue **RECHAZADO con error E11000** (impide que otros clientes compren ese producto).
Solución del equipo: Validación de esquema con `$jsonSchema` y `$expr` usando `$setUnion` (código en página 2).

Contraste Crítico 2: Precisión Monetaria y Persistencia de Totales

Afirmación IA: *'Guardar subtotal y total calculados con números estándar de punto flotante para acelerar lecturas'.*
Veredicto: PARCIALMENTE CORRECTA.
• **Lo incorrecto:** Usar coma flotante estándar. La documentación oficial (*Model Monetary Data*) exige **Decimal128** para evitar imprecisiones de redondeo binario acumulativo. Se persistió con `NumberDecimal`.
• **Lo correcto:** Guardar el `total` es válido como redundancia controlada si se audita. Como MongoDB no posee triggers relacionales, la aplicación garantiza la coherencia y un pipeline de agregación audita que `total = suma_lineas` (0 discrepancias). El subtotal se omite para evitar redundancia transitiva (se calcula con `$multiply`).

Consigna 7: Conclusión Técnica y Justificación de la Migración

Para Food Store conviene combinar ambos motores: mantener PostgreSQL como sistema de registro y usar MongoDB para consultar pedidos completos.

El primer criterio es el **patrón de acceso y la localidad de datos**. La operación más frecuente es leer el pedido completo. Con cliente y líneas embebidos, MongoDB lo devuelve en una sola lectura, mientras que PostgreSQL necesita dos JOIN entre tres tablas para la consulta c). Como cada pedido tiene pocas líneas (1 a 6), el documento no crece sin límite.

El segundo criterio es la **integridad**. PostgreSQL garantiza de forma declarativa las claves foráneas, `UNIQUE(id_pedido, id_producto)` y los `CHECK` de cantidad y precio. En MongoDB hubo que reconstruir esas reglas con un validador (`$jsonSchema` y `$setUnion`), y el total guardado es una redundancia que la aplicación debe mantener y auditar (dio 0 descuadres). Catálogo, stock y clientes cambian seguido y se relacionan entre sí, algo que el modelo relacional resuelve mejor.

Por eso no conviene reemplazar PostgreSQL: los pedidos se registran allí y se copian a MongoDB como documentos cerrados, con el precio histórico congelado (R4). Así se aprovecha la lectura rápida sin perder las garantías del motor relacional.

191 palabras